
¿CÓMO VAMOS A ENTRENAR LAS RN 2?

Fernanda Sobrino

2026

Gradientes

Problemas entrenando redes neuronales

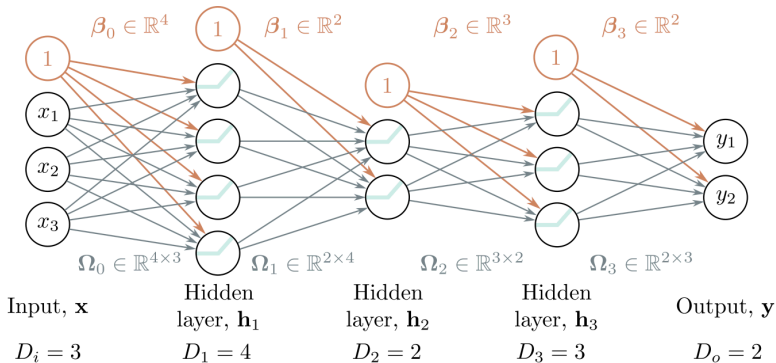
Para poder entrenar una red neuronal necesitamos:

1. una manera eficiente de calcular gradientes;
2. una buena manera de inicializar los parámetros;
3. un algoritmo de optimización estable.

En modelos grandes, el número de parámetros puede ser enorme, así que calcular gradientes a mano no es una opción práctica.

El problema con los gradientes

Consideremos la siguiente red:



El problema con los gradientes

Podemos escribir la red como una composición:

$$h_1 = a(\beta_0 + \Omega_0 x)$$

$$h_2 = a(\beta_1 + \Omega_1 h_1)$$

$$h_3 = a(\beta_2 + \Omega_2 h_2)$$

$$f(x, \phi) = \beta_3 + \Omega_3 h_3$$

El problema con los gradientes

Para entrenar necesitamos una pérdida:

$$L(\phi) = \sum_{i=1}^n \ell(f(x_i, \phi), y_i)$$

Y actualizamos usando mini-batch SGD:

$$\phi_{t+1} = \phi_t - \alpha \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_{\phi} \ell_i(\phi_t)$$

Por lo tanto necesitamos derivadas como $\frac{\partial \ell_i}{\partial \beta_k}$ y $\frac{\partial \ell_i}{\partial \Omega_k}$.

Gradientes

Las parciales de la pérdida nos dicen cómo cambia la pérdida cuando cambiamos un poco los parámetros.

En una red neuronal:

- ▶ cada parámetro afecta una transformación intermedia;
- ▶ esa transformación afecta a la siguiente capa;
- ▶ el efecto se propaga hasta la salida;
- ▶ la salida afecta la pérdida.

Para saber cómo un parámetro afecta la pérdida, necesitamos seguir todo ese camino.

Ese camino se calcula con la regla de la cadena.

Recordemos

Forward pass:

Recorremos la red hacia adelante. Dado x_i y los parámetros, calculamos:

$$\hat{y}_i = f(x_i, \phi)$$

y después la pérdida:

$$\ell(f(x_i, \phi), y_i)$$

Backward pass:

Recorremos la red hacia atrás usando la regla de la cadena para calcular los gradientes necesarios para actualizar los pesos.

Ejemplo sencillo

Consideremos la función:

$$f(x, \phi) = \beta_3 + \omega_3 \cos(\beta_2 + \omega_2 \exp(\beta_1 + \omega_1 \sin(\beta_0 + \omega_0 x)))$$

Podemos verla como una red pequeña con un input y un output.

Usamos:

$$\ell_i = (f(x_i, \phi) - y_i)^2$$

Ejemplo sencillo

Podríamos calcular todas las parciales a mano.

Pero:

- ▶ las expresiones se vuelven largas rápidamente;
- ▶ hay muchos cálculos repetidos;
- ▶ podemos reutilizar derivadas intermedias.

Backpropagation resuelve esto mediante:

1. forward pass: calcular y guardar valores intermedios;
2. backward pass: calcular gradientes de atrás hacia adelante.

Ejemplo sencillo: forward pass

Escribimos la función como una serie de pasos intermedios:

$$\begin{array}{ll} f_0 = \beta_0 + \omega_0 x_i & h_1 = \sin(f_0) \\ f_1 = \beta_1 + \omega_1 h_1 & h_2 = \exp(f_1) \\ f_2 = \beta_2 + \omega_2 h_2 & h_3 = \cos(f_2) \\ f_3 = \beta_3 + \omega_3 h_3 & \ell_i = (f_3 - y_i)^2 \end{array}$$

Ejemplo sencillo: backward pass

Empezamos desde la pérdida: $\frac{\partial \ell_i}{\partial f_3} = 2(f_3 - y_i)$.

Luego aplicamos la regla de la cadena:

$$\frac{\partial \ell_i}{\partial h_3} = \frac{\partial \ell_i}{\partial f_3} \frac{\partial f_3}{\partial h_3}$$

Y podemos seguir hacia atrás hasta llegar a:

$$\frac{\partial \ell_i}{\partial f_0} = \frac{\partial \ell_i}{\partial f_3} \frac{\partial f_3}{\partial h_3} \frac{\partial h_3}{\partial f_2} \frac{\partial f_2}{\partial h_2} \frac{\partial h_2}{\partial f_1} \frac{\partial f_1}{\partial h_1} \frac{\partial h_1}{\partial f_0}$$

Ejemplo sencillo: backward pass

Pero no queremos solo derivadas respecto a variables intermedias.

Queremos derivadas respecto a los parámetros:

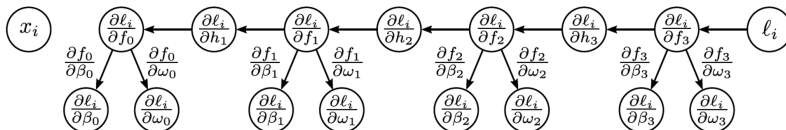
$$\frac{\partial \ell_i}{\partial \omega_k} \quad \frac{\partial \ell_i}{\partial \beta_k}$$

Usamos otra vez la regla de la cadena:

$$\frac{\partial \ell_i}{\partial \omega_k} = \frac{\partial \ell_i}{\partial f_k} \frac{\partial f_k}{\partial \omega_k}$$

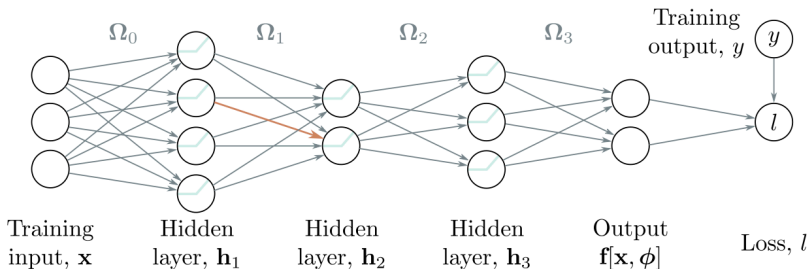
Esto es más eficiente que recalcular cada derivada desde cero.

Ejemplo sencillo: backward pass



Ejemplo vectorial/matricial de backprop

Tenemos la siguiente red:



Ejemplo vectorial/matricial de backprop

Forward pass:

$$\begin{aligned}f_0 &= \beta_0 + \Omega_0 x_i & h_1 &= a(f_0) \\f_1 &= \beta_1 + \Omega_1 h_1 & h_2 &= a(f_1) \\f_2 &= \beta_2 + \Omega_2 h_2 & h_3 &= a(f_2) \\f_3 &= \beta_3 + \Omega_3 h_3 & \ell_i &= \ell(f_3, y_i)\end{aligned}$$

Ejemplo vectorial/matricial de backprop

Definimos:

$$\delta_k = \frac{\partial \ell_i}{\partial f_k}$$

Para la última capa, calculamos δ_3 desde la pérdida.

Luego propagamos hacia atrás:

$$\delta_2 = (\Omega_3^T \delta_3) \odot a'(f_2)$$

Si a es ReLU: $a'(f_2) = \mathbf{1}[f_2 > 0]$.

Ejemplo vectorial/matricial de backprop

Nos faltan las derivadas respecto a los parámetros.

Para una capa:

$$f_k = \beta_k + \Omega_k h_k$$

las derivadas son:

$$\frac{\partial \ell_i}{\partial \beta_k} = \delta_k \quad \frac{\partial \ell_i}{\partial \Omega_k} = \delta_k h_k^T$$

Para la primera capa podemos tomar $h_0 = x_i$.

Algoritmo de propagación hacia atrás

Backpropagation calcula los gradientes moviéndose de la salida hacia el input.

Para cada capa:

$$\frac{\partial \ell_i}{\partial \beta_k} = \delta_k \quad \frac{\partial \ell_i}{\partial \Omega_k} = \delta_k h_k^T$$

$$\delta_{k-1} = (\Omega_k^T \delta_k) \odot a'(f_{k-1})$$

Si a es ReLU: $a'(f_{k-1}) = \mathbf{1}[f_{k-1} > 0]$.

Algoritmo de propagación hacia atrás

Este algoritmo es eficiente porque reutiliza derivadas intermedias.

Computacionalmente usa principalmente:

- ▶ multiplicaciones de matrices;
- ▶ operaciones elemento a elemento;
- ▶ acumulación de gradientes por mini-batch.

Pero requiere memoria.

Durante el forward pass tenemos que guardar activaciones intermedias para poder usarlas en el backward pass.

Diferenciación algorítmica

Los frameworks modernos calculan los gradientes automáticamente.

Durante el **forward pass**:

- ▶ calculan el output y la pérdida;
- ▶ guardan las operaciones y valores intermedios.



Durante el **backward pass**:

- ▶ recorren esas operaciones en sentido contrario;
- ▶ aplican la regla de la cadena;
- ▶ calculan los gradientes de todos los parámetros.



$$\text{forward} \rightarrow L \rightarrow \text{backward} \rightarrow \nabla_{\phi} L$$

Diferenciación algorítmica

Backpropagation es diferenciación algorítmica en **modo reverso**: $L \rightarrow$ capas finales $\rightarrow$ capas iniciales.

La red no tiene que ser una cadena simple. Puede tener:

- ▶ ramas;
- ▶ conexiones residuales;
- ▶ operaciones reutilizadas;
- ▶ recurrencia.

En una RNN, la recurrencia se puede “desenrollar” en el tiempo, $h_1 \rightarrow h_2 \rightarrow \dots \rightarrow h_T$, y después aplicar backpropagation sobre esta gráfica.

forward pass construye el cómputo	backward pass calcula los gradientes
-----------------------------------	--------------------------------------

Inicialización

¿Cómo inicializamos los parámetros?

El punto donde comenzamos afecta:

- ▶ estabilidad numérica;
- ▶ magnitudes de activaciones;
- ▶ magnitudes de gradientes;
- ▶ velocidad de entrenamiento.

Durante el forward pass:

$$f_k = \beta_k + \Omega_k h_k \quad h_{k+1} = a(f_k)$$

Si usamos ReLU e inicializamos $\beta_k = 0$ y $\Omega_{ij} \sim N(0, \sigma_\Omega^2)$, la elección de σ_Ω^2 es crucial.

¿Cómo inicializamos los parámetros?

Si σ_{Ω} es demasiado pequeño:

- ▶ las preactivaciones tienden a hacerse pequeñas;
- ▶ las activaciones pueden contraerse capa tras capa;
- ▶ los gradientes también pueden hacerse muy pequeños.



Si σ_{Ω} es demasiado grande:

- ▶ las preactivaciones pueden crecer capa tras capa;
- ▶ las activaciones pueden explotar;
- ▶ los gradientes también pueden explotar.



El objetivo es mantener escalas razonables hacia adelante y hacia atrás.

¿Por qué esto es un problema?

El problema aparece tanto en el forward pass como en el backward pass.

Si las activaciones o gradientes se vuelven extremadamente pequeños:

vanishing gradients

Si se vuelven extremadamente grandes:

exploding gradients

Ambos problemas dificultan la optimización.

Preservar la varianza hacia adelante

Consideremos una capa:

$$h = a(f) \quad f' = \beta + \Omega h$$

Supongamos $\beta_i = 0$ y $\Omega_{ij} \sim N(0, \sigma_\Omega^2)$.

Si f es simétrica alrededor de cero y a es ReLU, entonces aproximadamente:

$$E[h_j^2] \approx \frac{1}{2} \sigma_f^2$$

Preservar la varianza hacia adelante

Para la unidad i :

$$f'_i = \sum_{j=1}^{D_h} \Omega_{ij} h_j$$

Como los pesos tienen media cero, $E[f'_i] = 0$.

La varianza queda aproximadamente (con ReLU):

$$\sigma_{f'}^2 = D_h \sigma_{\Omega}^2 E[h_j^2] \approx \frac{1}{2} D_h \sigma_{\Omega}^2 \sigma_f^2$$

Inicialización He

Queremos que $\sigma_{f'}^2 \approx \sigma_f^2$, entonces necesitamos $\frac{1}{2}D_h\sigma_{\Omega}^2 \approx 1$.

Por lo tanto:

$$\sigma_{\Omega}^2 = \frac{2}{D_h}$$

Esta es la idea detrás de la inicialización He para ReLU.

Aquí D_h es el número de inputs de la capa, también llamado **fan-in**.

Inicialización y backward pass

Un razonamiento parecido se puede hacer para el backward pass.

Queremos que los gradientes tampoco exploten ni desaparezcan al propagarse hacia atrás.

La condición hacia atrás depende del número de outputs de la capa, llamado **fan-out**.

Si la capa no es cuadrada, preservar exactamente ambas varianzas al mismo tiempo puede no ser posible.

Por eso las inicializaciones prácticas son compromisos diseñados para mantener escalas razonables.

Xavier/Glorot initialization

Para activaciones aproximadamente simétricas como tanh, una inicialización clásica es Xavier/Glorot:

$$\sigma_{\Omega}^2 = \frac{2}{D_{in} + D_{out}}$$

donde D_{in} es el fan-in y D_{out} es el fan-out.

Para ReLU, normalmente se usa He initialization:

$$\sigma_{\Omega}^2 = \frac{2}{D_{in}}$$

Otras inicializaciones

La inicialización sigue siendo importante en Deep Learning.

Reglas prácticas útiles:

- ▶ ReLU / Leaky ReLU: empezar con He initialization.
- ▶ tanh / sigmoid: empezar con Xavier/Glorot.
- ▶ redes muy profundas: revisar si las activaciones y gradientes explotan o desaparecen.

No es magia negra, aunque a veces los tutoriales se empeñen en hacerla parecer astrología con matrices.

Desempeño

Desempeño

El error en test puede venir de varias fuentes:

1. ruido inherente del problema;
2. pocos ejemplos de entrenamiento;
3. modelo con capacidad insuficiente;
4. modelo con demasiada flexibilidad para los datos disponibles;
5. problemas de optimización;
6. diferencias entre la distribución de entrenamiento y la de prueba.

En Deep Learning queremos distinguir dos preguntas:

¿podemos optimizar el modelo? ¿el modelo generaliza?

Ruido, sesgo y varianza

Ruido: parte impredecible del proceso generador de datos.

Puede venir de:

- ▶ error de medición;
- ▶ variables no observadas;
- ▶ etiquetas erróneas;
- ▶ variabilidad inherente del fenómeno.

Este error no desaparece simplemente cambiando el modelo.

Ruido, sesgo y varianza

Sesgo: error sistemático porque la familia de modelos no puede representar bien la relación verdadera.

Intuición:

modelo demasiado simple $\Rightarrow$ alto sesgo

En redes neuronales, reducir sesgo suele implicar aumentar capacidad:

- ▶ más neuronas;
- ▶ más capas;
- ▶ mejores arquitecturas.

Ruido, sesgo y varianza

Varianza: sensibilidad del modelo al conjunto de entrenamiento.

Si entrenamos el mismo algoritmo con distintos datasets, podemos obtener modelos distintos.

La varianza puede venir de:

- ▶ tamaño finito del conjunto de entrenamiento;
- ▶ ruido en los datos;
- ▶ flexibilidad del modelo;
- ▶ aleatoriedad del algoritmo de entrenamiento.

Reduciendo la varianza

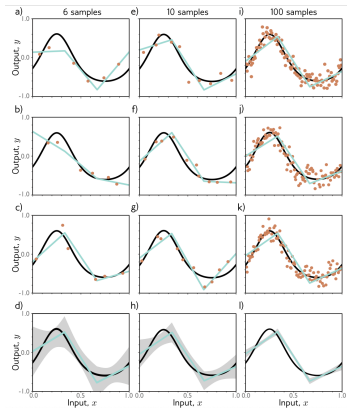
Una forma usual de reducir varianza es aumentar la cantidad de datos de entrenamiento.

Intuición:

$n \uparrow \Rightarrow$ menos dependencia de ejemplos particulares

Esto suele ayudar, pero no garantiza automáticamente mejor desempeño si los datos nuevos son ruidosos, sesgados o vienen de otra distribución.

Reduciendo la varianza



Reduciendo el sesgo

Para reducir sesgo necesitamos una familia de modelos más flexible.

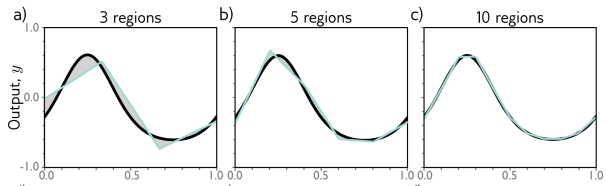
En redes neuronales esto puede significar:

- ▶ agregar neuronas;
- ▶ agregar capas;
- ▶ cambiar la arquitectura;
- ▶ usar mejores representaciones.

Pero aumentar capacidad también puede aumentar la varianza.

Ese es el trade-off clásico.

Reduciendo el sesgo



¿Qué pasa si reducimos demasiado el sesgo?

En la intuición clásica, aumentar demasiado la capacidad puede llevar a overfitting.

El modelo ajusta muy bien los datos de entrenamiento, incluyendo ruido o peculiaridades de la muestra.

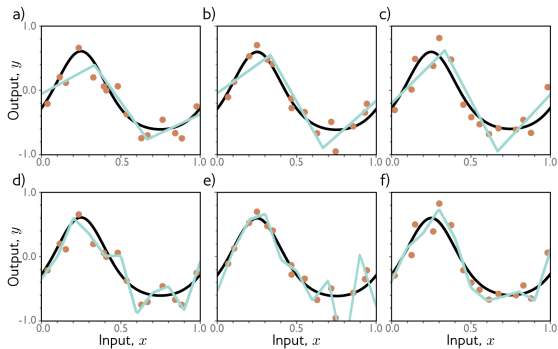
Entonces podemos observar:

$$R_{train} \downarrow$$

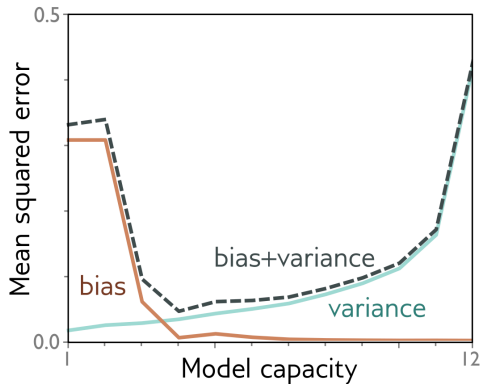
pero:

$$R_{test} \uparrow$$

¿Qué pasa si reducimos el sesgo? overfitting



Sesgo vs Varianza



¿Qué es double descent?

La intuición clásica dice:

más capacidad $\Rightarrow$ menos sesgo pero más riesgo de overfitting

Pero en modelos modernos muchas veces vemos otro patrón.

Al aumentar la capacidad:

1. baja el error de test;
2. luego sube cerca del umbral de interpolación;
3. después vuelve a bajar en el régimen sobreparametrizado.

A este patrón se le llama **double descent**.

¿Qué es double descent?

El umbral de interpolación ocurre cerca del punto donde el modelo apenas puede lograr:

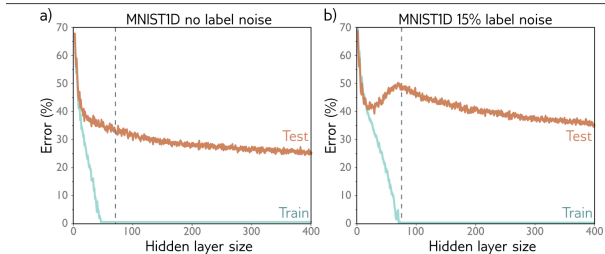
$$R_{train} \approx 0$$

Tenemos tres zonas:

- ▶ **subparametrizada:** el modelo no tiene suficiente capacidad;
- ▶ **cerca del umbral de interpolación:** puede ajustar los datos, pero puede ser inestable;
- ▶ **sobrep parametrizada:** hay muchas soluciones que interpolan los datos.

Ojo: double descent no es una ley universal. Depende de datos, ruido, arquitectura y optimización.

¿Qué es double descent?



¿Qué es double descent?

Intuición:

- ▶ En baja capacidad, el modelo no puede representar bien la función.
- ▶ Cerca del umbral de interpolación, puede ajustar los datos pero quizá de forma inestable.
- ▶ En sobreparametrización, existen muchas soluciones que ajustan entrenamiento.
- ▶ El algoritmo de optimización y la inicialización pueden sesgar la búsqueda hacia algunas soluciones y no otras.

La pregunta difícil es:

¿por qué algunas de esas soluciones generalizan bien?

¿Qué es double descent?

Double descent es desconcertante porque el error de test puede seguir mejorando incluso después de que el error de entrenamiento ya es casi cero.

Esto contradice la versión más simple de la historia clásica de sesgo-varianza.

Una idea importante es el **sesgo inductivo**:

el algoritmo y la arquitectura prefieren ciertas soluciones sobre otras

Aunque muchas soluciones interpolen los datos, no todas son igualmente probables de encontrar durante el entrenamiento.

La maldición de la dimensionalidad

En espacios de alta dimensión, los datos se vuelven escasos.

Aunque tengamos muchos ejemplos, el espacio posible de inputs puede ser enorme.

Intuición:

- ▶ los puntos de entrenamiento pueden estar muy alejados entre sí;
- ▶ muchas regiones del espacio no contienen ejemplos;
- ▶ generalizar requiere hacer supuestos sobre qué ocurre entre los puntos observados.

La maldición de la dimensionalidad

La maldición de la dimensionalidad no es una explicación completa de double descent.

Pero sí ayuda a entender por qué la generalización en alta dimensión es difícil.

En ausencia de datos en gran parte del espacio, el modelo necesita algún sesgo inductivo para decidir cómo interpolar o extrapolar.

En redes profundas, ese sesgo puede venir de:

- ▶ la arquitectura;
- ▶ la inicialización;
- ▶ el algoritmo de optimización;
- ▶ los datos usados para entrenar.

Double descent

Una intuición posible es que, en el régimen sobreparametrizado, existen muchas soluciones que ajustan los datos.

Entonces la pregunta no es solamente:

¿existe una solución?

sino:

¿qué tipo de solución encuentra el entrenamiento?

Hiperparámetros

¿Cómo los escogemos en la práctica?

Algunos hiperparámetros importantes son:

- ▶ número de capas;
- ▶ número de neuronas por capa;
- ▶ algoritmo de optimización;
- ▶ tasa de aprendizaje;
- ▶ batch size;
- ▶ número de épocas.

Normalmente se escogen de manera empírica usando el conjunto de validación.

¿Cómo los escogemos en la práctica?

El proceso típico es:

1. entrenar varios modelos con distintos hiperparámetros;
2. evaluarlos en el conjunto de validación;
3. escoger la configuración con mejor desempeño en validación;
4. evaluar una sola vez el modelo final en el conjunto de prueba.

Escoger hiperparámetros puede ser computacionalmente costoso.

No usamos el conjunto de prueba para tomar decisiones, porque eso contamina la evaluación final.